DigiLab: Digitaltechnik

Stand: Oktober 2026 · Version 9.126

Kurz: Nullen und Einsen, Gatter, Wahrheitstabellen, Addierer, Flipflops, Zähler, echte ICs und eine kleine CPU zum Programmieren. Alles zum Ausprobieren im DigiLab.

1. Analog und digital: Nullen und Einsen

In der **Digitaltechnik** kennt jedes Signal nur zwei Zustände: **0** (Low, meist nahe 0 V) und **1** (High, meist nahe der Versorgungsspannung, zum Beispiel 5 V). Alles dazwischen ist „verboten" und wird je nach Schwelle als 0 oder 1 gedeutet. Der Vorteil: Störungen und Rauschen machen nichts aus, solange der Pegel nicht auf die falsche Seite der Schwelle rutscht.

Begriff	Bedeutung
Bit	eine Stelle mit den Werten 0 oder 1
Nibble	vier Bit (ein Hexadezimalzeichen)
Byte	acht Bit
Pegel	Spannungsbereich für 0 und für 1
Schwelle	etwa die halbe Versorgungsspannung (CMOS)

> **Merke:** Im DigiLab liegen die Gatter an 5 V: Eine Leitung über 2,5 V gilt als 1, darunter als 0. Eine LED zeigt dir den Zustand.

2. Zahlensysteme: binär, hexadezimal, BCD

Rechner zählen mit den Stellenwerten 1, 2, 4, 8, 16 … (Zweierpotenzen). Die Binärzahl `10101101` bedeutet 128 + 32 + 8 + 4 + 1 = **173**. Vier Bit fasst man zu einer **Hexadezimalziffer** (0 bis 9, A bis F) zusammen: `10101101` = `AD` hex.

Dezimal	Binär	Hex	BCD
0	0000	0	0000
1	0001	1	0001
2	0010	2	0010
5	0101	5	0101
9	1001	9	1001
10	1010	A	0001 0000
15	1111	F	0001 0101

BCD (Binary Coded Decimal) kodiert jede Dezimalstelle einzeln mit vier Bit, die Zahl 73 ist also `0111 0011`. Negative Zahlen schreibt man als **Zweierkomplement**: alle Bits umkehren und eins addieren. Die 8-Bit-Zahl −5 ist `11111011`.

> **Merke:** Beim Umwandeln hilft der Zahlenumrechner im DigiLab (Menü) mit Übungsaufgaben.

3. Die Grundgatter und die Wahrheitstabelle

Ein **Gatter** verknüpft Eingänge zu einem Ausgang. Die **Wahrheitstabelle** listet alle Eingangskombinationen und das Ergebnis.

A	B	UND	ODER	NAND	NOR	XOR
0	0	0	0	1	1	0
0	1	0	1	1	0	1
1	0	0	1	1	0	1
1	1	1	1	0	0	0

- **UND (&):** 1 nur, wenn alle Eingänge 1 sind.
- **ODER (≥1):** 1, wenn mindestens ein Eingang 1 ist.
- **NICHT:** kehrt um.
- **NAND, NOR:** UND bzw. ODER mit nachgeschaltetem NICHT.
- **XOR (=1):** 1, wenn die Eingänge verschieden sind.

> **Merke: NAND** (und ebenso NOR) ist universell: Aus NAND-Gattern allein lässt sich jede Schaltung bauen.

Im DigiLab öffnest du im Menü die **Wahrheitstabelle**: Das Lab probiert alle Schalterkombinationen durch und zeigt dir die Tabelle der aktuellen Schaltung samt Gleichung.

4. Schaltalgebra und Vereinfachen

Mit den Regeln der **Schaltalgebra** (Boolesche Algebra) lassen sich Schaltungen berechnen und vereinfachen. Schreibweise: · bedeutet UND, + bedeutet ODER, ¬ bedeutet NICHT.

Regel	Gleichung
Identität	$A \cdot 1 = A$, $A + 0 = A$
Null und Eins	$A \cdot 0 = 0$, $A + 1 = 1$
Komplement	$A \cdot \neg A = 0$, $A + \neg A = 1$
Doppelte Verneinung	$\neg\neg A = A$
De Morgan	$\neg(A \cdot B) = \neg A + \neg B$, $\neg(A + B) = \neg A \cdot \neg B$

Absorption	$A + A \cdot B = A$

Aus einer Wahrheitstabelle gewinnt man die Gleichung als **Summe von Produkten**: Für jede Zeile mit Ergebnis 1 ein UND-Term, alle Terme mit ODER verknüpft. Dann vereinfacht man (zum Beispiel mit dem KV-Diagramm oder dem Quine-McCluskey-Verfahren). Beispiel XOR: $Y = \neg A \cdot B + A \cdot \neg B$.

> **Merke:** Das Werkzeug **Schaltung aus Tabelle** im DigiLab macht genau das: Du trägst die Tabelle ein, das Lab leitet die kürzeste Gleichung her und baut die Schaltung mit Schaltern und LEDs.

5. KV-Diagramm: Gruppen bilden

Das **KV-Diagramm** (Karnaugh-Veitch) ordnet die Zeilen einer Wahrheitstabelle so an, dass sich benachbarte Felder in genau einem Eingang unterscheiden (Gray-Code: 00, 01, 11, 10). Benachbarte Einsen lassen sich zu **Gruppen** zusammenfassen, und jede Gruppe wird ein einfacher UND-Term.

Regel	Erklärung
Gruppengröße	1, 2, 4, 8 oder 16 Felder (Zweierpotenzen), nur Einsen
möglichst groß	große Gruppen ergeben kurze Terme
alle Einsen abdecken	jede 1 gehört in mindestens eine Gruppe
Rand zählt als Nachbar	links und rechts, oben und unten hängen zusammen
Don’t-Care (X)	ein Feld, dessen Wert egal ist, darf in eine Gruppe, muss aber nicht

In der Gruppe fallen die Variablen weg, die in der Gruppe sowohl 0 als auch 1 sind; übrig bleiben die, die gleich bleiben. Alle Gruppen werden mit ODER verknüpft.

> **Merke:** Im Werkzeug „KV-Diagramm“ tippst du die Felder an (0, 1, X). Das Lab zeichnet die Gruppen farbig, liest die Gleichung ab und baut die Schaltung. „Aus Schaltung übernehmen“ holt die Funktion aus der aktuellen Wahrheitstabelle.

6. Rechenwerke: Halb- und Volladdierer

Zwei Bits addieren: 0+0=0, 0+1=1, 1+0=1, 1+1=**10** (Summe 0, Übertrag 1). Der **Halbaddierer** besteht aus einem XOR (Summe) und einem UND (Übertrag). Der **Volladdierer** berücksichtigt zusätzlich den Übertrag der Stelle davor: Summe = A xor B xor C_{in}, $C_{out} = A \cdot B + (A \text{ xor } B) \cdot C_{in}$.

A	B	Cin	Summe	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1

1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Mehrere Volladdierer hintereinander, bei denen der Übertrag von Stelle zu Stelle läuft, addieren Zahlen mit mehr Bit (Ripple-Carry-Addierer).

7. Speicher: Latch und Flipflop

Gatter allein haben kein Gedächtnis: Der Ausgang folgt sofort den Eingängen. Mit **Rückkopplung** (Ausgang zurück an einen Eingang) entsteht ein Speicher. Das einfachste Beispiel ist das **SR-Latch** aus zwei NOR-Gattern: S (Set) setzt Q auf 1, R (Reset) löscht Q, bei S = R = 0 bleibt der alte Zustand.

Das **D-Flipflop** übernimmt den Wert am Eingang D nur bei der steigenden **Taktflanke** und hält ihn bis zur nächsten. Das **T-Flipflop** kippt bei jeder Flanke, wenn T = 1. Das **JK-Flipflop** kennt Setzen (J), Löschen (K) und Kippen (beides 1).

Flipflop	Verhalten bei steigender Taktflanke
D	Q = D
T	T = 1: Q kippt, T = 0: Q bleibt
JK	J=1, K=0: Q = 1; J=0, K=1: Q = 0; J=K=1: Q kippt; J=K=0: Q bleibt

> **Achtung:** Flipflops brauchen eine definierte Taktflanke. Gerade beim Einschalten ist der Zustand zufällig: Mit einem Reset-Eingang bringt man sie in einen bekannten Anfangszustand.

8. Zähler, Schieberegister und das Zeitdiagramm

Schaltet man **T-Flipflops** hintereinander, wobei jedes den $\bar{Q}$-Ausgang des vorigen als Takt bekommt, halbiert jede Stufe die Frequenz: Es entsteht ein **Binärzähler**. Vier Stufen zählen von 0 bis 15 (Q0 ist das niedrigste Bit).

Im **Schieberegister** reicht jedes D-Flipflop sein Bit an das nächste weiter: Bei jeder Taktflanke rutscht der Inhalt eine Stelle.

Wie die Signale zeitlich zusammenhängen, zeigt das **Zeitdiagramm** (Logikanalysator): Tippe im Messen-Werkzeug auf mehrere Bauteile, dann erscheint jedes als eigene Spur. Beim Binärzähler siehst du die Teilerkette: Jede Spur ist halb so schnell wie die darüber.

> **Merke:** Frequenzteiler: Mit n T-Flipflops teilst du den Takt durch 2^n. Aus 4 Hz werden nach drei Stufen 0,5 Hz.

9. Zustandsautomaten und Reset

Ein **Zustandsautomat** (endlicher Automat) kennt eine endliche Zahl von **Zuständen**. Bei jedem Takt wechselt er je nach Eingang in einen Folgezustand, und seine **Ausgänge** hängen vom Zustand ab (Moore-Automat). Beispiele: Ampel, Treppenhauslicht, Erkennung einer Bitfolge.

Gebaut wird er aus **Zustandsspeicher** (D-Flipflops) und **Schaltnetz** (Gatter), das aus Zustand und Eingang den Folgezustand bildet. Die Schritte:

1. Zustandsdiagramm aufzeichnen,
2. Zustände binär codieren (n Zustände brauchen aufgerundet lb n Flipflops),
3. Folgezustandstabelle aufstellen,
4. Gleichungen für die D-Eingänge und die Ausgänge herleiten (zum Beispiel mit KV-Diagramm; unbenutzte Codes sind Don't-Care),
5. Schaltung bauen und prüfen.

> **Merke:** Das Werkzeug „Zustandsautomat" im DigiLab macht alle Schritte: Du trägst Zustände und Übergänge ein, siehst Diagramm, Tabelle und Gleichungen und lässt die Schaltung bauen. Mit den Takt-Knöpfen spielst du den Automaten vorher durch.

Ein **Reset-Eingang** bringt den Automaten in einen bekannten Zustand. Beim **synchronen Reset** wirkt er mit dem nächsten Takt (die D-Eingänge werden mit ¬R verundet). Beim **asynchronen Reset** wirkt er sofort, ohne auf den Takt zu warten: Beim 7474 sind das Clear (CLR) und Preset (PRE), aktiv bei Low.

Reset	wirkt	Beispiel
synchron	mit der nächsten Taktflanke	Reset-Schalter im Zustandsautomaten
asynchron	sofort	CLR und PRE am 7474, R0 am 7490

Mit den Reset-Eingängen kann man Zähler verkürzen: Ein Dezimalzähler 7490, dessen QB und QC auf R0 zurückgeführt werden, springt beim Wert 6 sofort auf 0 und zählt nur 0 bis 5.

10. Echte ICs: 74xx und 40xx

In der Praxis nimmt man fertige Bausteine: Die **74er-Reihe** (TTL, zum Beispiel 7400 mit vier NAND-Gattern) und die **40er-Reihe** (CMOS, zum Beispiel CD4011). Jeder IC hat Versorgungspins (VCC und GND bzw. VDD und VSS) und eine festgelegte Pinbelegung.

IC	Inhalt
CD4011 / 7400	vier NAND-Gatter
CD4017	Dezimalzähler mit zehn Ausgängen (Lauflicht)
7490	Dezimalzähler (BCD)
CD4511 / 7447	Decoder: BCD in Segmente für die 7-Segment-Anzeige

7474 / CD4013	zwei D-Flipflops

> **Achtung: Unbenutzte Eingänge niemals offen lassen.** CMOS-Eingänge „schweben" und liefern zufällige Pegel, TTL-Eingänge liegen offen auf High. Immer auf Plus oder Masse legen. Auch Schalter brauchen einen **Pull-down** (10 kΩ nach Masse), sonst ist der Pegel bei offenem Schalter unbestimmt.

11. Die Mini-CPU TF-4

Ein **Prozessor** ist im Kern ein riesiges Schaltwerk: Ein Befehlszähler (PC) holt Befehle aus dem Programmspeicher, ein Rechenwerk (ALU) führt sie aus, Register halten Zwischenergebnisse. Die **Mini-CPU TF-4** im DigiLab hat vier Bit Datenbreite, einen Akkumulator A, ein Register B, 16 Halbbytes RAM, vier Eingänge, vier Ausgänge und die Flags Z (Ergebnis null) und C (Übertrag).

Befehl	Wirkung
LDI n	A = n
OUT	Ausgänge = A
IN	A = Eingänge
INC / DEC	A um 1 hoch / runter
ADD	A = A + B
ADDI n	A = A + n
SHL	A um eine Stelle nach links, C = herausgeschobenes Bit
JMP marke	springt
JNZ / JNC marke	springt bei Z = 0 bzw. C = 0
CMPI n	vergleicht A mit n (Flags)

Beispielprogramm (Zähler):

```
start: LDI 0 loop: OUT INC JMP loop
```

Mit jeder steigenden Taktflanke führt die CPU genau **einen** Befehl aus. Im Programmfenster (Menü „Mini-CPU: Programm") siehst du Register, Flags, RAM und den nächsten Befehl und kannst Schritt für Schritt ausführen.

> **Merke:** Ein Programm ist nur eine Folge von Befehlen im Speicher. Genau wie beim Arduino ändert sich das Verhalten der Schaltung allein durch das Programm.

Unterprogramme: CALL und RET

Was du mehrfach brauchst, schreibst du nur einmal als **Unterprogramm**. `CALL marke` merkt sich die Adresse des nächsten Befehls auf einem **Stapel** und springt zur Marke, `RET` holt die Adresse zurück und springt dorthin. Die

Mini-CPU hat vier Stapelplätze: Unterprogramme dürfen sich also bis zu vier Ebenen tief aufrufen. Im Programmfenster siehst du den Stapel.

```
start: LDI 5 CALL zeige LDI 10 CALL zeige JMP start zeige: OUT NOP RET
```

> **Merke:** Das Unterprogramm steht hinter dem Hauptprogramm (nach JMP start), sonst läuft es versehentlich mit. Jedes CALL braucht ein RET, sonst wird der Stapel voll.

Zusätzlich gibt es `PUSH` und `POP`: A auf einen Datenstapel (vier Plätze) legen und wieder holen, zum Beispiel, um A in einem Unterprogramm zu sichern.

12. Arbeitsblatt: Digitaltechnik

Name: _____ Datum: _____ Punkte: _____ / 17

1. Wandle die Binärzahl 10101101 in Dezimal und Hexadezimal um.

2. Schreibe die Dezimalzahl 59 binär (8 Bit) und als BCD.

3. Wie lautet die 8-Bit-Zweierkomplementzahl für −12?

4. Vervollständige die Wahrheitstabelle für Y = NOR(A, B).

5. Zeichne oder nenne eine Schaltung, die ein NICHT-Gatter nur aus einem NAND-Gatter bildet.

6. Gegeben ist die Wahrheitstabelle Y = 1 genau für (A,B) = (0,1) und (1,0). Wie lautet die Gleichung? Welches Gatter ist das?

7. Vereinfache $Y = A \cdot B + A \cdot \neg B$.

8. Addiere binär 0110 + 0111. Welche Summe und welcher Übertrag entstehen (4 Bit)?

9. Was ist der Unterschied zwischen einem Gatter und einem Flipflop?

10. Ein 4-Bit-Binärzähler wird mit 8 Hz getaktet. Mit welcher Frequenz schaltet das höchste Bit (Q3)? Wie lange dauert ein voller Zählzyklus?

11. Nenne zwei Gründe, warum man unbenutzte Eingänge eines CMOS-Bausteins nicht offen lässt.

12. Die Mini-CPU führt aus: LDI 3, ADDI 5, OUT. Was steht danach im Ausgangsregister (4 Bit)? Was ist mit dem Übertrag C?

13. Beschreibe in zwei Sätzen, was dieses Programm macht: `LDI 1 / loop: OUT / SHL / JNC loop / JMP 0`

14. Zeichne die Zustandsfolge einer Ampel mit den Phasen Rot, Rot-Gelb, Grün, Gelb. Wie viele Flipflops braucht der Automat?

15. Was ist der Unterschied zwischen synchronem und asynchronem Reset?

16. Mit einem KV-Diagramm lassen sich die Einsen 0, 2, 8 und 10 (vier Variablen ABCD) zu einer Gruppe zusammenfassen. Wie lautet der Term?

17. Warum braucht ein Unterprogramm einen Stapel für die Rückkehradresse?